A Comparison of Machine Learning Models for Credit Card Default Prediction

Fazel Rabbi, Zoya Malik, and Siyi Ye
Department of Computer Science, University of Calgary, Calgary, AB, Canada

***Abstract*—This paper compared four supervised machine learning models (Logistic Regression, Decision Tree, K-Nearest Neighbours (KNN), and a Neural Network) for predicting credit card default risk using the Kaggle Credit Card Approval Prediction dataset. The dataset contained 36,457 applicant records and 17 features, with a severe class imbalance of 88.2 % low-risk and 11.8 % high-risk applicants. A binary target variable was constructed from monthly payment status history, where any record of a payment overdue by 30 or more days was labelled high-risk. Class imbalance was handled through class weighting and decision-threshold adjustment. Models were ranked using a composite score that weighted recall and ROC-AUC most heavily, since correctly identifying high-risk applicants was considered more important than overall accuracy. KNN ($K = 1$) ranked first with a composite score of 0.483, and Logistic Regression ranked second (0.421) due to its high recall. The results showed that simpler models can outperform more complex ones when the evaluation metric is tailored to the actual problem constraints.**

***Index Terms*—credit risk, binary classification, machine learning, class imbalance, k-nearest neighbours, logistic regression.**

I. Introduction

Predicting whether a credit card applicant will default is an important problem in consumer finance. Lenders need to identify high-risk applicants before approving them, because missing a high-risk applicant (a false negative) is typically more costly than incorrectly flagging a low-risk one [1]. Machine learning has become a common approach for this type of classification problem, as it can learn patterns from large applicant datasets more flexibly than traditional scorecard methods [2].

One of the main challenges in credit default prediction is class imbalance. In most real-world datasets, the majority of applicants are low-risk, which causes many models to simply predict low-risk for nearly every applicant and still achieve high accuracy. This makes accuracy a poor metric for evaluating model performance in this context, and evaluation methods need to reflect the actual cost of each type of error.

In this project, four models were trained and evaluated on an imbalanced credit dataset: Logistic Regression as a baseline, Decision Tree, KNN, and a Neural Network. A composite scoring formula was designed to prioritise recall and ROC-AUC over accuracy, reflecting the real-world importance of catching high-risk applicants. The goal was to determine which model performed best under these conditions.

The rest of the paper is organised as follows. Section II covers the dataset and methods used. Section III presents the results. Section IV discusses the findings and limitations.

II. Method

A. Dataset

The dataset used was the Kaggle Credit Card Approval Prediction dataset [3], which contained two files: `application_record.csv` (applicant demographic and financial information) and `credit_record.csv` (monthly payment status history). The two files were joined on applicant ID using an inner join, and 47 duplicate records were removed, resulting in 36,457 unique records and 17 features including income, employment duration, housing type, family size, and education level.

The target variable was constructed from the `STATUS` field in the credit record file. An applicant was labelled high-risk (1) if any monthly status code of 1, 2, 3, 4, or 5 appeared in their history, meaning they had at least one payment overdue by 30 or more days. All other applicants were labelled low-risk (0). The final class distribution was 88.2 % low-risk and 11.8 % high-risk.

B. Preprocessing

A shared preprocessing script produced a single `cleaned_data.csv` file used by all four models. Missing values in `OCCUPATION_TYPE` were filled using the mode. The `DAYS_BIRTH` and `DAYS_EMPLOYED` columns were converted to `AGE` and `YEARS_EMPLOYED` in years, and the sentinel value 365,243 in `DAYS_EMPLOYED` (indicating self-employment or unemployment) was set to zero. Categorical columns were one-hot encoded, and numerical columns were standardised using scikit-learn's `StandardScaler` [4]. The scaler was fit only on the training set and applied to the test set to prevent data leakage. An 80/20 stratified train–test split was used across all models (`random_state = 42`).

C. Models

Logistic Regression was used as the baseline. `class_weight='balanced'` was set so the model would account for the minority class, and `max_iter` was set to 1,000.

Decision Tree was trained with a maximum depth of 5. Depths from 2 to 15 were tested, and depth 5 was selected as the point where recall stabilised without a significant drop in accuracy (Fig. 1). `class_weight='balanced'` was also applied.

Neural Network was a Sequential model with two hidden layers (`Dense(64, relu)` and `Dense(32, relu)`) and

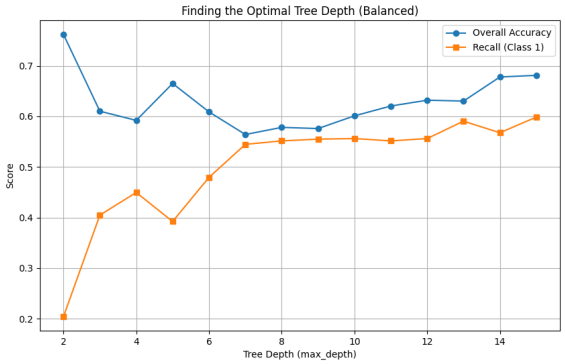

Fig. 1. Decision Tree accuracy and minority-class recall across depths 2–15. Depth 5 was chosen where recall plateaued relative to further depth increases.

a `Dense(1, sigmoid)` output. It was compiled with the Adam optimiser and binary cross-entropy loss, and trained for 20 epochs with batch size 32. The decision threshold was lowered to 0.30 (from the default 0.50) to improve recall on the imbalanced data.

KNN was evaluated for K from 1 to 20. The best K was selected by maximising a combined criterion of $0.5 \times \text{ROC-AUC} + 0.5 \times \text{Recall}$ on the test set. The selection process is shown in Fig. 5, and $K = 1$ was identified as the optimal value.

D. Evaluation Metrics

All models were evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. A composite score was also calculated:

$$S = 0.35\,R + 0.30\,A_{\text{ROC}} + 0.20\,F_1 + 0.10\,P + 0.05\,A \quad (1)$$

where R is Recall, A_{ROC} is ROC-AUC, F_1 is F1-Score, P is Precision, and A is Accuracy. Recall received the highest weight because missing a high-risk applicant is the most costly outcome. Accuracy received the lowest weight since it is misleading under class imbalance.

III. Results

Table I reports all five evaluation metrics and the composite score for each model on the test set ($n = 7{,}292$), ranked by composite score. Fig. 4 shows a visual comparison of Recall, F1-Score, ROC-AUC, and composite score across models.

KNN ($K = 1$) achieved the highest composite score of 0.483. Logistic Regression ranked second with 0.421. Decision Tree ranked third with 0.398, and the Neural Network ranked last with 0.397. The Neural Network had the highest accuracy (85.78 %) and ROC-AUC (0.663). Logistic Regression had the highest recall (49.18 %). KNN had the highest precision (38.52 %) and F1-Score (38.02 %).

Fig. 2 shows the confusion matrices for all four models. On the test set of 860 actual high-risk applicants, Logistic Regression identified the most with TP = 423 and FN = 437, but also produced the most false positives (FP = 2,701). KNN correctly identified 323 high-risk applicants (FN = 537), with far fewer false positives (FP = 514). The Neural Network identified the fewest (TP = 176, FN = 684), and Decision Tree fell between them: TP = 337, FN = 523, FP = 1,884.

Fig. 3 shows the ROC curve for Logistic Regression (AUC = 0.549), which was only marginally above the random-guess baseline of 0.50.

TABLE I
Model Performance on the Test Set ($n = 7{,}292$)

Model	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)	AUC
KNN ($K = 1$)†	85.60	**38.52**	37.53	38.02	0.648
Log. Reg.	57.06	13.54	**49.18**	21.23	0.549
Dec. Tree	67.00	15.17	39.22	21.88	0.561
Neural Net.	**85.78**	33.15	20.51	**25.34**	**0.663**

† Highest composite score: 0.483. Full composite scores: KNN 0.483, LR 0.421, DT 0.398, NN 0.397. Bold = best value per column.

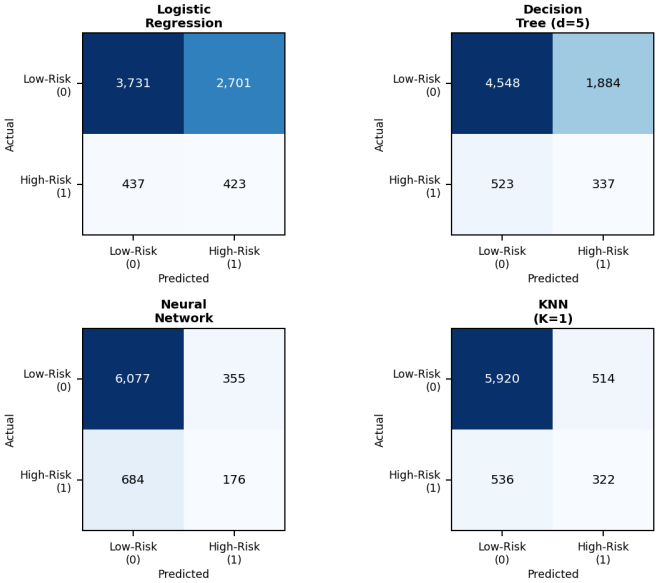

Fig. 2. Confusion matrices for all four models on the test set. Rows = actual class, columns = predicted class.

IV. Discussion

A. Summary

This project compared four models on a binary credit default classification task with severe class imbalance (88.2 % vs. 11.8 %). KNN ($K = 1$) ranked first on the composite metric, followed by Logistic Regression, Decision Tree, and the Neural Network. Overall, all models struggled to achieve high recall on the minority class, which reflects both the difficulty of the class imbalance problem and the limitations of the dataset.

B. Model Interpretation

KNN ($K = 1$) ranking above the Neural Network was a surprising result, since neural networks are generally expected

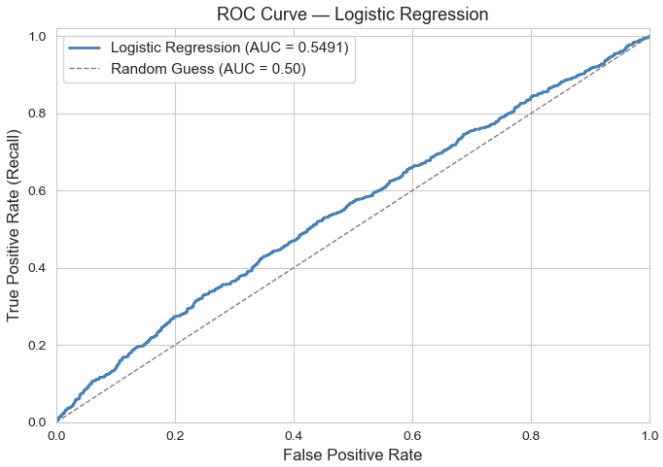

Fig. 3. ROC curve for Logistic Regression (AUC = 0.549). The curve lies only slightly above the random-guess diagonal, reflecting the difficulty of the classification task.

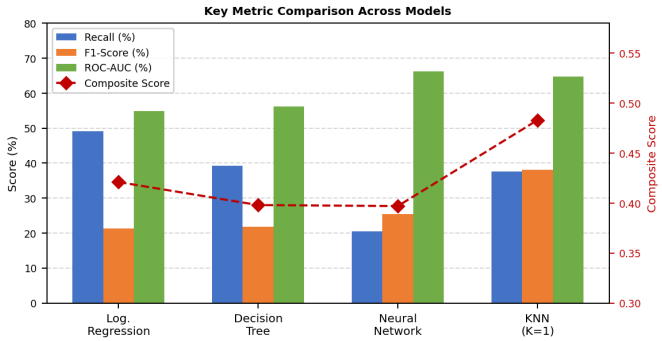

Fig. 4. Comparison of Recall, F1-Score, ROC-AUC (%), and composite score across all four models. Composite score on right axis.

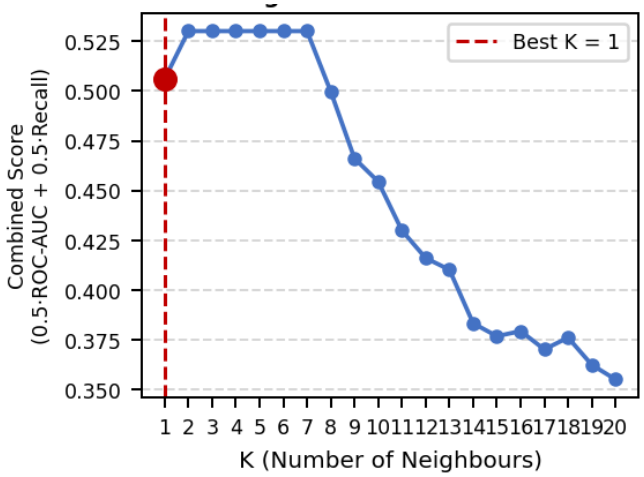

Fig. 5. KNN combined score (0.5×ROC-AUC + 0.5×Recall) for $K = 1$ to 20. $K = 1$ was selected as the optimal value by the algorithm.

to perform better on tabular data. However, the Neural Network did not use class weighting, so even with the lowered threshold of 0.30, it still skewed toward predicting the majority class. Its recall of 20.51 % was the lowest of all four models, which

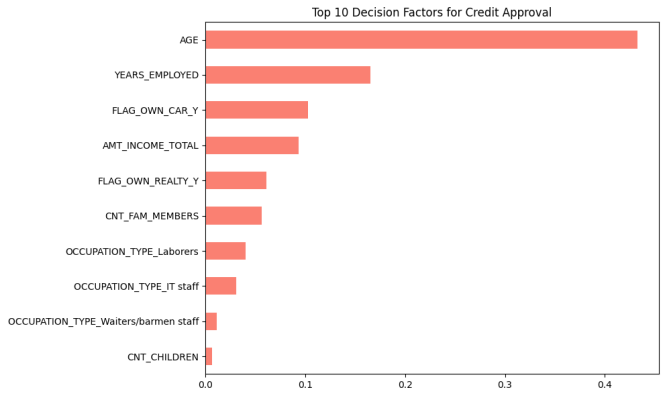

Fig. 6. Top 10 feature importances from the Decision Tree ($d = 5$). AGE and YEARS_EMPLOYED dominate the model's splitting decisions.

heavily penalised it under the composite formula.

Logistic Regression performed well on recall because `class_weight='balanced'` pushed it to predict high-risk more often. However, this came at the cost of a large number of false positives (2,701), which brought its precision down to 13.54 %.

The Decision Tree with max depth 5 produced moderate results across all metrics. As shown in Fig. 6, AGE and YEARS_EMPLOYED were by far the most important features, together accounting for over 60% of the tree's splitting decisions. This is consistent with the credit-scoring literature, where financial stability indicators are known to be strong predictors of risk [1]. Limiting the depth helped prevent overfitting, but also restricted the model from capturing more complex interactions in the feature space.

C. Influence of the Dataset

The way the target variable was constructed affected all model results. Any applicant with even a single late payment (status 1–5) was labelled high-risk, which grouped together borderline cases with severe defaulters. This likely introduced label noise and made the classification boundary harder to learn.

The dataset also only contained demographic and application data (income, family size, housing type, etc.) with no behavioural features such as credit utilisation or spending history. These types of features are typically much more predictive of default behaviour, and their absence likely explains why all models had relatively low ROC-AUC values.

One methodological inconsistency was also found: the Neural Network one-hot encoded categorical features before the train–test split, while all other models encoded inside the training pipeline only. Although the effect of this is minimal for one-hot encoding, it is still a data leakage issue and should be fixed in future work.

D. Limitations

Only a single train–test split was used, so it is not possible to say how stable the results would be across different partitions. Class imbalance was addressed only through class weighting

and threshold adjustments; more advanced resampling techniques like SMOTE [5] were not tested. KNN with $K = 1$ is sensitive to noisy data, and its top ranking may not hold with different random seeds. The composite formula weights were also chosen manually without a formal cost analysis, which is a limitation of the evaluation method.

E. Future Improvements

Using k-fold cross-validation instead of a single split would give more reliable and stable performance estimates. Testing data-level resampling like SMOTE [5] alongside ensemble models such as Random Forest or Gradient Boosting could meaningfully improve minority-class detection, as shown in similar studies [2]. Tuning the Neural Network threshold more carefully using the Youden J statistic on a validation set, rather than setting it manually to 0.30, would also improve its recall without changing the architecture. Including behavioural or transactional features in the dataset would also likely improve performance across all models. Finally, applying feature selection before training could help reduce noise from less informative features, which may be especially beneficial for KNN since it relies on distance calculations across all input dimensions.

CONCLUSION

This project showed that model selection in imbalanced classification problems is heavily influenced by how performance is measured. KNN ($K = 1$) ranked first under a recall-weighted composite metric, while the Neural Network ranked last despite achieving the highest accuracy and ROC-AUC. The results highlight that no single metric tells the full story, and that evaluation criteria should be designed to reflect the actual cost structure of the problem being solved.

DATA AVAILABILITY

The dataset is publicly available on Kaggle: kaggle.com/datasets/rikdifos/credit-card-approval-prediction [3]. All project code is available at: github.com/DATA221-Final-Project.

Author Contributions

- **Zoya Malik:** Data preprocessing, Logistic Regression model, report writing.
- **Fazel Rabbi:** Neural Network and KNN models, visualizations.
- **Siyi Ye:** Decision Tree model, depth tuning, feature importance analysis.

References

[1] D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: a review," *J. R. Stat. Soc., Ser. A*, vol. 160, no. 3, pp. 523–541, 1997.

[2] I. Brown and C. Mues, "An experimental comparison of classification algorithms for imbalanced credit scoring data sets," *Expert Syst. Appl.*, vol. 39, no. 3, pp. 3446–3453, 2012.

[3] Rikdifos, "Credit Card Approval Prediction Dataset," Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.